MODULE 15

HTML IN FULL STACK

THEORY ASSIGNMENT

1. HTML Basics:

1.Define HTML. What is the purpose of HTML in web development?

HTML stands for **HyperText Markup Language**.
It is the standard language used to create and structure web pages.

HTML uses different **tags** (like <html>, <head>, <body>, <p>, <h1>) to define elements on a webpage such as headings, paragraphs, images, links, tables, forms, etc.

Purpose of HTML in Web Development

HTML is used to build the **basic structure (skeleton)** of a website.

Main Purposes of HTML:

1. **Structure Web Pages**
    - HTML organizes content into headings, paragraphs, sections, tables, forms, etc.
2. **Display Content**
    - It tells the browser what content should appear on the webpage.
3. **Create Links**
    - HTML allows navigation between pages using hyperlinks (<a> tag).
4. **Add Media**
    - Images, videos, audio, and other multimedia can be added using HTML tags.
5. **Create Forms**
    - HTML forms collect user data like login, registration, feedback, etc.
6. **Work with CSS and JavaScript**
    - HTML works together with:
        - CSS → for styling/design
        - JavaScript → for functionality and interactivity

Simple Example of HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <p>This is a simple webpage.</p>
</body>
</html>
```

2.Explain the basic structure of an HTML document. Identify the mandatory tags and their purposes ?

Basic Structure of an HTML Document

An HTML document has a fixed basic structure that helps the browser understand and display the webpage correctly.

Basic HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is an HTML document.</p>
</body>
</html>
```

Mandatory Tags and Their Purposes

1. <!DOCTYPE html>

Purpose:

- Declares that the document is an HTML5 document.
- Helps the browser display the webpage correctly.

Example:

```
<!DOCTYPE html>
```

Hindi:

Ye browser ko batata hai ki document HTML5 me likha gaya hai.

---

2. <html> Tag

Purpose:

- Root element of an HTML page.
- All HTML code is written inside this tag.

Example:

```
<html>
  ...
</html>
```

---

3. <head> Tag

Purpose:

- Contains information about the webpage.
- Stores metadata, title, CSS links, JavaScript links, etc.

Example:

```
<head>
  <title>My Page</title>
</head>
```

---

4. <title> Tag

Purpose:

- Defines the title of the webpage.
- The title appears on the browser tab.

Example:

```
<title>My Website</title>
```

---

5. <body> Tag

Purpose:

- Contains all visible content of the webpage.
- Text, images, headings, links, forms, etc. are written inside it.

Example:

```
<body>
  <h1>Hello</h1>
</body>
```

3. What is the difference between block-level elements and inline elements in HTML? Provide examples of each ?

Difference Between Block-Level Elements and Inline Elements in HTML

HTML elements are mainly divided into two types:

1. **Block-Level Elements**
2. **Inline Elements**

---

1. Block-Level Elements

Definition

Block-level elements always start on a **new line** and take the **full width** available.

They create a separate block on the webpage.

Features of Block-Level Elements

- Starts from a new line
- Takes full width
- Can contain other block and inline elements
- Used for page structure/layout

Examples of Block-Level Elements

<h1>This is Heading</h1>
This is Paragraph
This is Div

Common block-level tags:

-
-
- <h1> to <h6>
- <section>
- <article>
-
-

Output Behavior

Heading

Paragraph

Div

Each element appears on a separate line.

---

2. Inline Elements

Definition

Inline elements do **not** start on a new line.
They only take as much width as necessary.

Features of Inline Elements

- Does not start on a new line
- Takes only required width
- Usually used inside block elements
- Cannot normally contain block elements

Examples of Inline Elements

Hello
Click Here
Bold Text

Common inline tags:

-
- <a>
-
-
- <img>
- <label>

Output Behavior

Hello Click Here Bold Text

All elements appear in the same line.

Main Difference Table

Feature	Block-Level Elements	Inline Elements
Starts on new line	Yes	No
Width	Full width	Required width only
Structure	Creates blocks	Flows within text
Can contain	Inline & block elements	Mostly text/inline elements
Example	, <p>	, <a>

Simple Example Together

```
<div>
  This is a block element
</div>

<span>This is inline element</span>
<a href="#">Link</a>
```

4. Discuss the role of semantic HTML. Why is it important for accessibility and SEO? Provide examples of semantic elements.

Role of Semantic HTML

Semantic HTML means using HTML tags that clearly describe the meaning and purpose of the content.

Semantic elements help:

- Browsers
- Search engines
- Developers
- Screen readers

understand the structure of a webpage properly.

What is Semantic HTML?

Semantic tags describe **what the content is**, not just how it looks.

Example:

<header>
<nav>
<section>
<article>
<footer>

These tags clearly explain the purpose of each section.

Non-Semantic vs Semantic

Non-Semantic Elements

These do not tell anything about the content.

Semantic Elements

<header>
<main>
<footer>

These explain the meaning of the content.

Importance of Semantic HTML

1. Better Accessibility

Accessibility means making websites usable for everyone, including disabled users.

Screen readers used by visually impaired users can understand semantic tags more easily.

Example:

- <nav> tells screen readers that this section contains navigation links.
- <main> identifies the main content area.

This improves user experience for disabled users.

2. Better SEO (Search Engine Optimization)

Search engines like Google understand webpage structure better with semantic tags.

Semantic HTML helps search engines:

- Understand important content
- Index pages correctly
- Improve ranking in search results

3. Easier Code Maintenance

Semantic tags make code:

- Cleaner
- More readable
- Easier to maintain

Developers can quickly understand webpage sections

Common Semantic HTML Elements

Semantic Element	Purpose
<header>	Top section/header of webpage
<nav>	Navigation links
<main>	Main content
<section>	Group of related content
<article>	Independent content/article
<aside>	Sidebar content
<footer>	Bottom/footer section
<figure>	Images/illustrations
<figcaption>	Caption for figure

Example of Semantic HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Semantic HTML</title>
</head>
<body>

<header>
  <h1>My Website</h1>
</header>

<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
</nav>

<main>
  <section>
```

```
    <article>
      <h2>HTML Basics</h2>
      <p>Semantic HTML improves structure.</p>
    </article>
  </section>
</main>

<footer>
  <p>Copyright 2026</p>
</footer>

</body>
</html>
```

Advantages of Semantic HTML

- Improves accessibility
- Improves SEO
- Better webpage structure
- Easier maintenance
- Cleaner and understandable code

2.HTML Forms:

1. What are HTML forms used for? Describe the purpose of the input, textarea, select, and button elements.

HTML forms are used to collect user input on a webpage.

Forms help users send data to a server such as:

- Login information
- Registration details
- Feedback
- Search data
- Contact forms

Basic Form Syntax

```
<form>
  Form Elements
</form>
```

The <form> tag is used to create a form.

Main Form Elements

1. <input> Element

Purpose

The <input> element is used to take user input.

It supports many types such as:

- text
- password
- email
- number
- checkbox
- radio button

Example

<input type="text" placeholder="Enter your name">

Common Input Types

Type	Purpose
text	Normal text input
password	Hidden password
email	Email input
number	Numeric input
checkbox	Multiple selections
radio	Single selection

---

2. <textarea> Element

Purpose

Used for multi-line text input.

Mostly used for:

- Comments
- Feedback
- Messages
- Descriptions

Example

```
<textarea rows="4" cols="30">
Enter message
</textarea>
```

---

3. <select> Element

Purpose

Used to create a dropdown list.

Options are created using the <option> tag.

Example

```
<select>
  <option>India</option>
  <option>USA</option>
  <option>Canada</option>
</select>
```

---

4. <button> Element

Purpose

Used to create clickable buttons.

Buttons are mainly used to:

- Submit forms
- Reset forms
- Perform actions with JavaScript

Example

```
<button>Submit</button>
```

Complete Form Example

```
<form>

  <label>Name:</label>
  <input type="text"><br><br>

  <label>Message:</label>
  <textarea></textarea><br><br>

  <label>Country:</label>
  <select>
    <option>India</option>
    <option>USA</option>
  </select><br><br>

  <button type="submit">Submit</button>

</form>
```

Summary Table

Element	Purpose
<form>	Creates form
<input>	Takes user input
<textarea>	Multi-line text input
<select>	Dropdown list
<button>	Clickable button

---

2. Explain the difference between the GET and POST methods in form submission. When should each be used?

Difference Between GET and POST Methods in HTML Forms

GET and POST are HTTP methods used to send form data from the client to the server.

They are used inside the <form> tag using the method attribute.

---

1. GET Method

Definition

The GET method sends form data through the URL.

Data becomes visible in the browser address bar.

Example

```
<form method="GET">
  <input type="text" name="username">
  <button type="submit">Submit</button>
</form>
```

URL Example

example.com/?username=Gopal

The data is attached to the URL.

Features of GET Method

- Data visible in URL
- Limited data size
- Less secure
- Faster for small requests
- Can be bookmarked
- Used mainly for fetching/searching data

Uses of GET

- Search forms
- Filters
- Public data requests
- URL sharing

---

2. POST Method

Definition

The POST method sends data inside the HTTP request body.

Data is not visible in the URL.

Example

```
<form method="POST">
  <input type="password" name="password">
  <button type="submit">Submit</button>
</form>
```

Features of POST Method

- Data hidden from URL
- More secure than GET
- No size limitation (practically large data supported)
- Cannot be bookmarked easily
- Used for sensitive or large data

Uses of POST

- Login forms
- Registration forms
- File uploads
- Payment forms
- Database updates

Main Difference Table

Feature	GET	POST
Data Location	URL	Request body
Visibility	Visible	Hidden
Security	Less secure	More secure
Data Size	Limited	Large data supported
Bookmarking	Possible	Not possible
Usage	Fetch data	Send/store data

Example Comparison

GET

website.com/?name=Gopal

Data visible in URL.

POST

website.com/

Data hidden internally.

When to Use GET

Use GET when:

- Data is not sensitive
- You want to search or retrieve data
- URL sharing/bookmarking is needed

Example:

- Search engine query
- Product filtering

When to Use POST

Use POST when:

- Data is sensitive
- Large amount of data is sent
- Data must be stored or updated

Example:

- Login form
- Signup form
- Payment system

3. What is the purpose of the label element in a form, and how does it improve accessibility?

Purpose of the <label> Element in HTML Forms

The <label> element is used to define a label or description for form elements such as:

- input fields
- checkboxes

- radio buttons
- textareas
- select menus

It helps users understand what information they need to enter.

Syntax of <label>

```
<label for="username">Username:</label>
<input type="text" id="username">
```

Purpose of the <label> Element

1. Describes Form Fields

The label tells the user what the input field is for.

Example:

```
<label>Name:</label>
<input type="text">
```

Here, "Name" describes the textbox.

---

2. Connects Text with Input Field

The for attribute of <label> connects it with an input field using the input's id.

Example:

```
<label for="email">Email:</label>
<input type="email" id="email">
```

---

3. Makes Forms Easier to Use

When users click the label text, the related input field automatically becomes active.

This is especially useful for:

- checkboxes
- radio buttons

Example with Checkbox

```
<label for="agree">I Agree</label>
<input type="checkbox" id="agree">
```

If the user clicks "I Agree", the checkbox gets selected.

How <label> Improves Accessibility

Accessibility means making websites usable for everyone, including disabled users.

The <label> element improves accessibility in many ways.

---

1. Helps Screen Readers

Screen readers used by visually impaired users can read labels correctly.

They tell users:

- what the field is
- what information is required

---

2. Improves User Experience

Users can click on the label text instead of clicking only the small input area.

This makes forms easier to use.

---

3. Better Form Navigation

Assistive technologies can properly identify relationships between labels and form controls.

Complete Example

```
<form>

  <label for="name">Name:</label>
  <input type="text" id="name"><br><br>

  <label for="email">Email:</label>
  <input type="email" id="email"><br><br>

  <label for="gender">Gender:</label>
  <select id="gender">
    <option>Male</option>
    <option>Female</option>
  </select>

</form>
```

Advantages of <label>

Benefit	Description
Better accessibility	Helps disabled users
Easy identification	Describes form fields
Improved usability	Clicking label activates field
Better screen reader support	Reads form correctly

3.HTML Tables :

1. Explain the structure of an HTML table and the purpose of each of the following elements:

<table>,<tr>,<td>, and <thead>.

HTML tables are used to display data in the form of rows and columns.

A table is created using different table-related tags.

Basic Structure of HTML Table

```
<table>

  <thead>
    <tr>
      <td>Name</td>
      <td>Age</td>
    </tr>
  </thead>

  <tr>
    <td>Gopal</td>
    <td>22</td>
  </tr>

</table>
```

Purpose of Table Elements

---

1. <table> Element

Purpose

The <table> tag is the main container used to create a table.

It holds:

- rows
- columns
- table data

Example

```
<table>
  ...
</table>
```

---

2. <tr> Element

Purpose

<tr> stands for **Table Row**.

It is used to create rows inside the table.

Example

```
<tr>
  ...
</tr>
```

---

3. <td> Element

Purpose

<td> stands for **Table Data**.

It is used to store actual data inside table cells.

Example

```
<td>Gopal</td>
<td>22</td>
```

---

4. <thead> Element

Purpose

<thead> is used to group the header section of the table.

Usually contains column headings.

Example

```
<thead>
  <tr>
    <td>Name</td>
    <td>Age</td>
  </tr>
</thead>
```

Complete Table Example

```
<table border="1">

   <thead>
      <tr>
         <td>Name</td>
         <td>Course</td>
         <td>Age</td>
      </tr>
   </thead>

   <tr>
      <td>Gopal</td>
      <td>MCA</td>
      <td>22</td>
   </tr>

   <tr>
      <td>Rahul</td>
      <td>BCA</td>
      <td>21</td>
   </tr>

</table>
```

Output Structure

```
-------------------------
| Name  | Course | Age |
-------------------------
| Gopal | MCA    | 22  |
| Rahul | BCA    | 21  |
-------------------------
```

Summary Table

Element	Purpose
<table>	Creates the table
<tr>	Creates a row
<td>	Stores table data
<thead>	Defines table header

2. What is the difference between colspan and rowspan in tables? Provide examples.

Difference Between colspan and rowspan in HTML Tables

In HTML tables, colspan and rowspan attributes are used to merge table cells. They help in organizing table data in a better and more structured format.

---

1. colspan

The colspan attribute is used to merge two or more columns into a single cell. It makes a cell span horizontally across multiple columns.

Syntax:

```
<td colspan="2">Content</td>
```

Example:

```
<table border="1">
  <tr>
    <th colspan="2">Student Information</th>
  </tr>
  <tr>
    <td>Name</td>
    <td>Gopal</td>
  </tr>
</table>
```

Explanation:

In this example, the heading “Student Information” occupies two columns instead of one.

---

2. rowspan

The rowspan attribute is used to merge two or more rows into a single cell. It makes a cell span vertically across multiple rows.

Syntax:

<td rowspan="2">Content</td>

Example:

```
<table border="1">
  <tr>
    <th rowspan="2">Name</th>
    <td>Gopal</td>
  </tr>
  <tr>
    <td>Rahul</td>
  </tr>
</table>
```

Explanation:

In this example, the “Name” cell occupies two rows vertically.

Difference Between colspan and rowspan

colspan	**rowspan**
Used to merge columns	Used to merge rows
Works horizontally	Works vertically
Spans left to right	Spans top to bottom
Useful for table headings	Useful for grouped row data

3. Why should tables be used sparingly for layout purposes? What is a better alternative?

Why Should Tables Be Used Sparingly for Layout Purposes?

In earlier web development, HTML tables were commonly used to design webpage layouts. However, tables should now be used only for displaying tabular data, not for page layouts.

Using tables for layout creates several problems:

- Tables make the HTML code complex and difficult to maintain.
- Webpages built with tables load more slowly because the browser must render the entire table before displaying content.
- Table-based layouts are not responsive and do not adapt well to different screen sizes like mobile devices.
- They reduce accessibility for screen readers and assistive technologies.
- Mixing layout and data structure makes the code less semantic and less organized.

Therefore, tables should be used sparingly and only when displaying data in rows and columns, such as marksheets, schedules, or reports.

Better Alternative

A better alternative for webpage layout is using CSS (Cascading Style Sheets), especially:

- **CSS Flexbox**
- **CSS Grid**

These modern layout techniques provide:

- Responsive design
- Cleaner and more maintainable code
- Faster page rendering
- Better alignment and positioning
- Improved accessibility and flexibility

Conclusion

Tables are suitable for displaying tabular data but should not be used for webpage layouts. Modern CSS techniques like Flexbox and Grid are better alternatives because they create responsive, flexible, and cleaner web designs.

MODULE 15

CSS IN FULL STACK

THEORY ASSIGNMENT

4. CSS Selectors & Styling:

1. What is a CSS selector? Provide examples of element, class, and ID selectors ?

A CSS selector is used to select HTML elements so that styles can be applied to them.
Selectors tell the browser which HTML element should receive the CSS styling.

In simple words, a selector “targets” HTML elements.

Types of CSS Selectors

1. Element Selector

An element selector selects HTML elements by their tag name.

Example:

```
<p>This is a paragraph</p>
```

```
p{
  color: blue;
}
```

Explanation:

- Here p is the selector.
- It applies the style to all <p> elements.

---

2. Class Selector

A class selector selects elements using the class attribute.
It is represented using a dot (.).

Example:

```
<p class="text">Hello World</p>
```

```
.text{
  color: red;
}
```

Explanation:

- .text is a class selector.

- It styles all elements having class="text".

---

3. ID Selector

An ID selector selects an element using the id attribute.
It is represented using a hash (#).

Example:

```
<h1 id="title">Welcome</h1>
```

```
#title{
  color: green;
}
```

Explanation:

- #title is an ID selector.
- It styles the element having id="title".

Difference Between Them

Selector	Symbol	Used For
Element Selector	tag name	Selects HTML elements
Class Selector	.	Selects multiple elements with same class
ID Selector	#	Selects one unique element

Conclusion

CSS selectors are used to target HTML elements for styling. Element selectors target tags, class selectors target classes, and ID selectors target unique elements.

2. Explain the concept of CSS specificity. How do conflicts between multiple styles get resolved?

Concept of CSS Specificity

CSS specificity is the rule used by browsers to decide which CSS style will be applied when multiple styles target the same HTML element.

In simple words, specificity determines the priority of CSS selectors.

If different CSS rules conflict with each other, the selector with higher specificity gets applied.

Order of CSS Specificity

CSS selectors have different priority levels:

Selector Type	Priority
Inline CSS	Highest
ID Selector (#id)	High
Class Selector (.class)	Medium
Element Selector (p, h1)	Low

Example of Specificity

HTML

```
<p id="text" class="para">Hello World</p>
```

CSS

```
p{
  color: blue;
}

.para{
  color: red;
}

#text{
  color: green;
}
```

Result:

The text color will be **green** because the ID selector has the highest specificity among these selectors.

How Conflicts Between Multiple Styles are Resolved

When multiple CSS rules apply to the same element, conflicts are resolved using the following order:

---

1. Inline CSS Wins First

Example:

```
<p style="color: orange;">Hello</p>
```

Inline CSS has the highest priority.

---

2. Higher Specificity Wins

- ID selector overrides class selector.
- Class selector overrides element selector.

Example:

```
p{
  color: blue;
}

.text{
  color: red;
}
```

If <p class="text"> is used, the color becomes red because class selector has higher specificity.

---

3. Last Rule Wins

If two selectors have the same specificity, the CSS rule written later is applied.

Example:

```
p{
  color: blue;
}

p{
  color: green;
}
```

Result: Green color will apply because it is written later.

Important Point: !important

The !important rule overrides normal specificity rules.

Example:

```
p{
  color: blue !important;
}
```

This style gets highest priority.

Conclusion

CSS specificity decides which style is applied when multiple styles target the same element. Inline styles have the highest priority, followed by ID selectors, class selectors, and element selectors. If specificity is equal, the last written rule is applied.

3. What is the difference between internal, external, and inline CSS? Discuss the advantages and disadvantages of each approach ?

Difference Between Internal, External, and Inline CSS

CSS can be added to HTML in three different ways:

1. Inline CSS
2. Internal CSS
3. External CSS

Each method is used for styling webpages, but they differ in usage, advantages, and disadvantages.

---

1. Inline CSS

Inline CSS is written directly inside an HTML element using the style attribute.

Example

```
<p style="color: red;">Hello World</p>
```

Advantages

- Easy to use for small changes.
- Useful for testing quick styles.
- Highest priority among normal CSS rules.

Disadvantages

- Makes HTML code messy.
- Difficult to maintain for large projects.
- Repeated code increases page size.
- Cannot reuse styles easily.

---

2. Internal CSS

Internal CSS is written inside the <style> tag within the <head> section of an HTML document.

Example

```
<head>
  <style>
    p{
      color: blue;
    }
  </style>
</head>
```

Advantages

- Styles are stored in one place inside the page.
- Better than inline CSS for small websites.
- No need for separate CSS file.

Disadvantages

- CSS works only for one webpage.
- Cannot reuse styles across multiple pages.
- Large styles make the HTML file bigger.

---

3. External CSS

External CSS is written in a separate .css file and linked to the HTML document.

Example

HTML File

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

CSS File (style.css)

```
p{
  color: green;
}
```

Advantages

- Keeps HTML code clean.
- CSS can be reused across many webpages.
- Easier maintenance and updates.
- Best approach for large websites.
- Improves website consistency.

Disadvantages

- Requires an extra CSS file.
- Website styling may not load if the file link is incorrect.
- Slightly more complex for beginners.

Difference Between Them

Feature	Inline CSS	Internal CSS	External CSS
Location	Inside HTML element	Inside <style> tag	Separate .css file
Reusability	No	Limited	Yes
Maintenance	Difficult	Moderate	Easy
Best For	Small changes	Single page styling	Large websites
Priority	Highest	Medium	Lowest among these

Conclusion

Inline CSS is useful for quick styling, internal CSS is suitable for small single-page websites, and external CSS is the best method for large and professional websites because it provides reusable, clean, and maintainable code.

5.CSS Box Model:

1. Explain the CSS box model and its components (content, padding, border, margin). How does each affect the size of an element?
CSS Box Model

The CSS box model is a concept used to describe the structure and spacing of HTML elements on a webpage.

In CSS, every HTML element is treated as a rectangular box.
This box consists of four main components:

1. Content
2. Padding
3. Border
4. Margin

These components together determine the total size and spacing of an element.

Components of CSS Box Model

1. Content

The content area is the actual part where text, images, or other data is displayed.

Example:

width: 200px;
height: 100px;

Effect on Size:

- Defines the main width and height of the element.
- It is the innermost part of the box.

---

2. Padding

Padding is the space between the content and the border.

Example:

padding: 20px;

Effect on Size:

- Increases the inner space around content.
- Adds extra size inside the border.
- Background color is visible in padding area.

---

3. Border

Border surrounds the padding and content.

Example:

border: 2px solid black;

Effect on Size:

- Adds thickness around the element.
- Increases total element size.

---

4. Margin

Margin is the outermost space outside the border.

Example:

margin: 15px;

Effect on Size:

- Creates space between elements.
- Does not affect the inner content area.
- Transparent by default.

Structure of Box Model

Margin
└── Border
    └── Padding
        └── Content

Example of Complete Box Model

```
<div class="box">Hello</div>
```

```
.box{
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 15px;
}
```

Total Size Calculation

If:

- Width = 200px
- Padding = 20px
- Border = 5px

Then total width becomes:

$$200 + 20 + 20 + 5 + 5 = 250\text{px}$$

Margin adds extra outer spacing but is usually not included in the actual element width.

Conclusion

The CSS box model defines how elements are structured and spaced on a webpage. The content holds the actual data, padding creates inner spacing, border surrounds the element, and margin creates outer spacing between elements. Together, they determine the final size and layout of an element.

2. What is the difference between border-box and content-box box-sizing in CSS?Which is the default?
Difference Between border-box and content-box in CSS

The box-sizing property in CSS defines how the total width and height of an element are calculated.

There are two main values of box-sizing:

1. content-box
2. border-box

---

1. content-box

content-box is the default box-sizing value in CSS.

In this model:

- Width and height apply only to the content area.
- Padding and border are added outside the specified width and height.

Example

```
div{
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: content-box;
}
```

Total Width Calculation

$$200 + 20 + 20 + 5 + 5 = 250\text{px}$$

Explanation

- Actual width becomes 250px.
- Padding and border increase the total size.

---

2. border-box

In border-box:

- Width and height include content, padding, and border.
- The total size remains fixed.

Example

```
div{
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
```

Explanation

- Total width stays 200px.
- Content area automatically shrinks to fit padding and border inside the width.

Main Difference

content-box	border-box
Padding and border are added outside width	Padding and border included inside width
Element size increases	Element size remains fixed
Difficult for layout management	Easier for responsive design
Default value	Commonly used in modern websites

Which is the Default?

The default value of box-sizing in CSS is:

box-sizing: content-box;

Conclusion

content-box calculates width and height only for the content area, while border-box includes padding and border within the specified size. content-box is the default CSS behavior, but border-box is preferred in modern web development because it makes layouts easier to manage.

6.CSS Flexbox :

1. What is CSS Flexbox, and how is it useful for layout design? Explain the terms flex-container and flex-item ?

What is CSS Flexbox?

CSS Flexbox (Flexible Box Layout) is a modern CSS layout method used to arrange, align, and distribute elements efficiently inside a container.

It helps developers create responsive and flexible webpage layouts without using floats or tables.

Flexbox makes it easier to:

- Align items
- Create responsive layouts
- Control spacing between elements
- Arrange items in rows or columns

Why is Flexbox Useful for Layout Design?

Flexbox is useful because it provides:

- Easy horizontal and vertical alignment

- Flexible spacing between elements
- Responsive design for different screen sizes
- Better control over element positioning
- Simplified layout creation

It is commonly used for:

- Navigation bars
- Centering content
- Card layouts
- Responsive sections
- Menus and galleries

Flex Container

A flex container is the parent element that holds flex items.

To create a flex container, the display: flex; property is used.

Example

```
.container{
  display: flex;
}
```

Explanation

- The element with display: flex becomes the flex container.
- All direct child elements automatically become flex items.

Flex Item

Flex items are the child elements inside a flex container.

They can:

- Grow or shrink
- Be aligned easily
- Change order and spacing

Example

```
<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
```

```
  <div>Item 3</div>
</div>
```

Here:

- .container → Flex Container
- Item 1, Item 2, Item 3 → Flex Items

Example of Flexbox

```
<div class="container">
  <div class="box">One</div>
  <div class="box">Two</div>
  <div class="box">Three</div>
</div>
```

```
.container{
  display: flex;
  gap: 10px;
}

.box{
  padding: 20px;
  border: 1px solid black;
}
```

Result

The boxes appear in a horizontal row with spacing between them.

Difference Between Flex Container and Flex Item

Flex Container	Flex Item
Parent element	Child element
Holds flex items	Exists inside container
Uses display: flex	Controlled by flex properties
Manages layout	Participates in layout

Conclusion

CSS Flexbox is a powerful layout system used for creating flexible and responsive webpage designs. The parent element is called the flex container, and its child elements are called flex items. Flexbox simplifies alignment, spacing, and layout management in modern web development.

2. Describe the properties justify-content, align-items, and flex-direction used inFlexbox ?

Flexbox Properties in CSS

1. flex-direction

The flex-direction property defines the direction in which flex items are placed inside a flex container. It controls whether the items are arranged in rows or columns.

Values:

- row → Items are placed horizontally from left to right (default).
- row-reverse → Items are placed horizontally from right to left.
- column → Items are placed vertically from top to bottom.
- column-reverse → Items are placed vertically from bottom to top.

Example:

```
.container{
  display: flex;
  flex-direction: row;
}
```

Purpose:

It helps developers control the layout direction of elements in a flexible and responsive way.

---

2. justify-content

The justify-content property is used to align flex items along the **main axis**. It controls the spacing and positioning of items horizontally (when flex-direction is row).

Values:

- flex-start → Items align at the beginning of the container.
- flex-end → Items align at the end of the container.
- center → Items align at the center.
- space-between → Equal space between items.
- space-around → Equal space around items.
- space-evenly → Equal spacing on all sides.

Example:

```
.container{
  display: flex;
  justify-content: center;
}
```

Purpose:

It is mainly used to manage spacing and alignment of items inside a flex container.

---

3. align-items

The align-items property is used to align flex items along the **cross axis**. It controls vertical alignment when the flex direction is row.

Values:

- stretch → Items stretch to fill the container (default).
- flex-start → Items align at the top.
- flex-end → Items align at the bottom.
- center → Items align at the center vertically.
- baseline → Items align according to text baseline.

Example:

```
.container{
  display: flex;
  align-items: center;
}
```

Purpose:

It helps control the vertical positioning and alignment of flex items inside the container.

Conclusion

The properties flex-direction, justify-content, and align-items are important features of Flexbox. They help in creating responsive and well-aligned layouts by controlling the direction, spacing, and alignment of flex items inside a container.

7. CSS Grid:

1. Explain CSS Grid and how it differs from Flexbox. When would you use Grid over Flexbox?

CSS Grid and Flexbox

What is CSS Grid?

CSS Grid is a two-dimensional layout system in CSS used to design web pages in rows and columns. It allows developers to create complex and responsive layouts easily.

In Grid layout, there are:

- **Grid Container** → Parent element
- **Grid Items** → Child elements inside the container

Example:

```
.container{
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

Here:

- display: grid enables Grid layout.
- grid-template-columns creates 3 columns.
- gap adds spacing between grid items.

Difference Between CSS Grid and Flexbox

CSS Grid	Flexbox
Grid is a **two-dimensional** layout system.	Flexbox is a **one-dimensional** layout system.
It works with both rows and columns together.	It works in either a row or a column at one time.
Best for complete page layouts.	Best for aligning items in a single direction.
Provides better control over large layouts.	Provides better control for smaller components.
Uses grid lines, rows, and columns.	Uses main axis and cross axis.

When to Use Grid Over Flexbox?

CSS Grid should be used when:

- A webpage requires both rows and columns.
- Creating complete page layouts.
- Designing dashboards, galleries, or complex structures.
- Equal spacing and positioning are needed in two directions.

Example Uses of Grid:

- Website page layout
- Photo gallery
- Admin dashboard
- Magazine-style design

When to Use Flexbox?

Flexbox should be used when:

- Items need alignment in a single row or column.
- Creating navigation bars, buttons, menus, or small UI sections.
- Centering elements easily.

Example Uses of Flexbox:

- Navbar
- Button alignment
- Card alignment
- Menu items

Conclusion

CSS Grid and Flexbox are both powerful CSS layout systems. Grid is mainly used for two-dimensional layouts involving rows and columns, while Flexbox is used for one-dimensional layouts. Grid is preferred for complex webpage structures, whereas Flexbox is ideal for simple alignment and spacing of items.

2. Describe the grid-template-columns, grid-template-rows, and grid-gap properties. Provide examples of how to use them ?
CSS Grid Properties

1. grid-template-columns

The grid-template-columns property is used to define the number and size of columns in a CSS Grid container.

It specifies how wide each column should be.

Example:

```
.container{
  display: grid;
  grid-template-columns: 200px 200px 200px;
}
```

Explanation:

- Creates 3 columns.
- Each column width is 200px.

Another Example:

```
.container{
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
}
```

Explanation:

- fr means “fraction of available space”.
- The middle column gets more space than the others.

Purpose:

It helps organize content into multiple vertical sections.

---

2. grid-template-rows

The grid-template-rows property is used to define the number and size of rows in a Grid container.

It controls the height of rows.

Example:

```
.container{
  display: grid;
  grid-template-rows: 100px 150px 200px;
}
```

Explanation:

- Creates 3 rows.
- Heights are 100px, 150px, and 200px.

Another Example:

```
.container{
  display: grid;
  grid-template-rows: 1fr 2fr;
}
```

Explanation:

- Second row gets twice the space of the first row.

Purpose:

It helps manage vertical layout and spacing of grid items.

---

3. grid-gap (or gap)

The grid-gap property is used to add spacing between rows and columns in a Grid layout.

Modern CSS commonly uses gap instead of grid-gap.

Example:

```
.container{
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```

Explanation:

- Adds 20px space between all rows and columns.

Separate Row and Column Gap:

```
.container{
  display: grid;
  row-gap: 10px;
  column-gap: 20px;
}
```

Purpose:

It improves readability and creates proper spacing between grid items.

Complete Example

```
.container{
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 100px 100px;
  gap: 15px;
}
```

Explanation:

- Creates 3 columns of equal width.
- Creates 2 rows of 100px height.
- Adds 15px spacing between items.

Conclusion

The grid-template-columns property defines column sizes, grid-template-rows defines row sizes, and gap adds spacing between grid items. These properties are essential for creating structured and responsive layouts using CSS Grid.

8. Responsive Web Design with Media Queries:

1.What are media queries in CSS, and why are they important for responsive design?
Media Queries in CSS

Media queries in CSS are used to apply different styles to a webpage based on the screen size, device type, or resolution. They help make websites responsive so that the layout looks good on desktops, tablets, and mobile devices.

Media queries allow developers to change the design according to different screen conditions.

Syntax of Media Query

```
@media screen and (max-width: 768px){
  body{
    background-color: lightblue;
  }
}
```

Explanation:

- @media → Used to create a media query.
- screen → Targets screen devices.
- max-width: 768px → Applies styles when screen width is 768px or smaller.

Example of Responsive Design

```
.container{
  display: flex;
}

@media screen and (max-width: 600px){
  .container{
    flex-direction: column;
  }
}
```

Explanation:

- On large screens, items appear in a row.
- On small screens, items change into a column layout.

Importance of Media Queries in Responsive Design

1. Makes Websites Mobile-Friendly

Media queries help websites adjust properly on mobile phones and tablets.

2. Improves User Experience

Users can easily read and navigate content on any device.

3. Supports Different Screen Sizes

Different CSS styles can be applied for desktops, laptops, tablets, and mobiles.

4. Increases Responsiveness

Layouts automatically adapt to available screen space.

5. Reduces Need for Separate Mobile Websites

One responsive website can work on all devices.

Common Media Query Breakpoints

Device Type	Screen Width
Mobile	max-width: 600px
Tablet	max-width: 768px
Laptop	max-width: 1024px
Desktop	min-width: 1200px

Types of Media Features

Feature	Purpose
max-width	Applies style below a specific width
min-width	Applies style above a specific width
orientation	Detects portrait or landscape mode
max-height	Applies style based on screen height

Conclusion

Media queries are an important feature of CSS used to create responsive web designs. They allow webpages to adapt to different screen sizes and devices, improving usability and user experience across mobiles, tablets, and desktops.

2. Write a basic media query that adjusts the font size of a webpage for screens smaller than 600px ?

```
body{
  font-size: 20px;
}

@media screen and (max-width: 600px){
  body{
    font-size: 14px;
  }
}
```

Explanation:

- The default font size is 20px.
- When the screen width becomes 600px or smaller, the font size changes to 14px.
- This helps improve readability on smaller devices like mobile phones.

9. Typography and Web Fonts:

1. Explain the difference between web-safe fonts and custom web fonts. Why might you use a web-safe font over a custom font?

Difference Between Web-Safe Fonts and Custom Web Fonts

1. Web-Safe Fonts

Web-safe fonts are fonts that are already installed on most operating systems and devices. These fonts do not need to be downloaded from the internet because they are commonly available on users' computers.

Examples of Web-Safe Fonts:

- Arial
- Times New Roman
- Verdana
- Georgia
- Courier New

Example:

```
body{
  font-family: Arial, sans-serif;
}
```

Features:

- Fast loading speed
- Supported on almost all devices
- No need to import font files

---

2. Custom Web Fonts

Custom web fonts are fonts that are not usually installed on devices. They are downloaded from external sources or font files using CSS.

Developers often use services like:

- Google Fonts
- Adobe Fonts

Example:

```
@import url('https://fonts.googleapis.com/css2?family=Roboto&display=swap');

body{
```

```
  font-family: 'Roboto', sans-serif;
}
```

Features:

- Provides unique and attractive designs
- Improves website appearance and branding
- Requires internet download

Difference Between Web-Safe Fonts and Custom Fonts

Web-Safe Fonts	Custom Web Fonts
Already installed on devices	Need to be downloaded
Faster loading	May slow page loading
Limited design choices	Many design styles available
Highly compatible	Depends on browser support
No internet required	Internet needed to load font

Why Use a Web-Safe Font Over a Custom Font?

1. Faster Website Loading

Web-safe fonts load quickly because they are already available on the user's system.

2. Better Compatibility

They work consistently across different browsers and operating systems.

3. Improved Performance

No extra font files need to be downloaded, reducing page size.

4. Reliable Display

The website text appears correctly even if internet speed is slow.

5. Simplicity

They are easy to use and require less setup.

Conclusion

Web-safe fonts are pre-installed fonts that provide better speed and compatibility, while custom web fonts offer more creative and modern designs. Web-safe fonts are preferred

when performance, simplicity, and universal support are important, whereas custom fonts are used when unique styling and branding are needed.

2. What is the font-family property in CSS? How do you apply a custom Google Font to a webpage ?

font-family Property in CSS

The font-family property in CSS is used to specify the font style of text on a webpage. It allows developers to choose which font should be displayed for an element.

A browser will use the first font if available. If it is not available, it will use the next font in the list.

Syntax

```
selector{
  font-family: font-name;
}
```

Example of font-family

```
body{
  font-family: Arial, sans-serif;
}
```

Explanation:

- Arial is the main font.
- sans-serif is the fallback font family if Arial is unavailable.

Types of Font Families

Font Family	Description
serif	Fonts with small lines at the edges
sans-serif	Simple fonts without extra lines
monospace	Each character has equal width
cursive	Handwriting-style fonts
fantasy	Decorative fonts

How to Apply a Custom Google Font

Google Fonts provides free custom fonts that can be added to webpages.

Step 1: Add Google Font Link

Add the font link inside the <head> section of HTML.

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
rel="stylesheet">
```

Step 2: Use the Font in CSS

```
body{
  font-family: 'Roboto', sans-serif;
}
```

Complete Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
rel="stylesheet">

  <style>
    body{
      font-family: 'Roboto', sans-serif;
    }
  </style>
</head>

<body>
  <h1>Hello World</h1>
</body>
</html>
```

Conclusion

The font-family property is used to define the text font in CSS. Custom Google Fonts can be added by importing the font link from Google Fonts and then applying it using the font-family property. This helps create attractive and modern webpage designs.

****